

Experiment 4

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t child1, child2;
```

```
    int status;
```

```
    // Create first child
```

```
    child1 = fork();
```

```
    if (child1 < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
        return 1;
```

```
}
```

```
if (child1 == 0)
```

```
{
```

```
    printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
```

```
    sleep(3);
```

```
    printf("Child 1: Completed\n");
```

```
    return 10;
```

```
}
```

```
// Create second child
```

```
child2 = fork();
```

```
child2 = fork();
```

```
if (child2 < 0)
```

```
{
```

```
    perror("fork failed");
```

```
    return 1;
```

```
}
```

```
if (child2 == 0)
```

```
{
```

```
    printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
```

```
    sleep(5);
```

```

    printf("Child 2: Completed\n");

    return 20;
}

// Parent waits using wait()

printf("Parent: PID = %d\n", getpid());

printf("Parent: Waiting for any child using wait()...\n");


pid_t finished = wait(&status);


printf("Parent: Child with PID %d completed.\n", finished);


if (WIFEXITED(status))
{
    printf("Child exit status: %d\n", WEXITSTATUS(status));
}
}

// Parent waits for specific child using waitpid()

printf("Parent: Waiting for Child 2 using waitpid()...\n");


waitpid(child2, &status, 0);


printf("Parent: Child 2 completed using waitpid().\n");

```

```

if (WIFEXITED(status))
{
    printf("Child 2 exit status: %d\n", WEXITSTATUS(status));
}

printf("Parent: All children completed.\n");

return 0;
}

```

```

ashwithreddy_earla@ASHWITHREDDYPC:~$ ./experiment4
Parent: PID = 820
Parent: Waiting for any child using wait()...
Child 2: PID = 822, PPID = 820
Child 1: PID = 821, PPID = 820
Child 1: Completed
Parent: Child with PID 821 completed.
Child exit status: 10
Parent: Waiting for Child 2 using waitpid()...
Child 2: Completed
Parent: Child 2 completed using waitpid().
Child 2 exit status: 20
Parent: All children completed.

```